Московский государственный университет имени М.В. Ломоносова

Направление 38.03.01 Экономика

Программа «Национальная экономика»

Определение кредитного скоринга заемщиков банка на основе технологии машинного обучения

Курсовая работа

Студент

Касенова Асем Ардаковна

Научный руководитель:

Кандидат экономических наук, доцент

Сидоренко Владимир Николаевич

Москва

2025

ОГЛАВЛЕНИЕ

Введение 3

Глава 1. Основы кредитного скоринга 6
1.1 Методы машинного обучения 6
1.1.1 KNN 6

Глава 2. Практическая реализация алгоритмов и результаты 9
2.1 Описание данных и процесс их подготовки 9
2.2 Реализация алгоритмов 11
2.2.1 Реализация алгоритма KNN 12
2.2.2 Точность KNN на оптимальных гиперпараметрах 13
2.3 Деревья решений 18
2.3.1 Реализация алгоритма 19

Заключение 23

Список литературы 24

Приложение А. Алгоритм KNN и настройка гиперпараметров 26

Приложение Б. Обработка данных и оценка моделей 27

Введение

Актуальность темы исследования. В настоящее время в финансовых учреждениях накапливается большое количество последовательных данных, включая истории транзакций, кредитные истории и истории взаимодействия с клиентами. Одно из ключевых направлений машинного обучения (далее МО) является разработка моделей, позволяющих анализировать широкий спектр задач, которые актуальны для банковского сектора.

Наиболее востребованной задачей, которую решают специалисты в области *Data Science* является задача кредитного скоринга. Кредитный скоринг – важный инструмент, позволяющий минимизировать риски, связанные с предоставлением кредитов. Несмотря на стремление многих финансовых организаций расширять свою деятельность за пределы традиционного банковского сектора, для них кредитование по-прежнему остается основным источником дохода. В связи с этим любое повышение точности оценки платежеспособности клиентов может быстро привести к увеличению прибыли банка за счет роста объемов кредитования при сохранении неизменного уровня риска.

Степень разработанности темы исследования Кредитный скоринг является классической задачей банковской сферы, которую решают на протяжении многих десятилетий. Идея заключается в том, что перед предоставлением существенной суммы денежных средств заемщику, с которым банк ранее не работал, необходимо оценить потенциальные риски. Риск для банка устанавливается с помощью понятия дефолта.

Существуют различные определения дефолта, однако основная суть заключается в том, что банк в течение N месяцев после выдачи кредита наблюдает за платежной дисциплиной клиента, и если в течение непрерывного промежутка в K дней клиент не производит обязательный платеж, то такая ситуация рассматривается как дефолт. При этом изменение значений для N и K приводит к разным вариантам определения дефолта, при этом в каждом банке он может определяться по-своему.

При разработке модели машинного обучения необходимо использовать только те данные, которые доступны на момент подачи клиентом заявки на кредит, исключая информацию, появившуюся позднее. Такой подход необходим, чтобы избежать обучения модели на той информации, которая не будет доступна в процессе реального скоринга.

Кроме того, при решении задач кредитного скоринга данные кредитных историй обладают некоторыми преимуществами по сравнению с данными транзакционных историй. Во-первых, кредитная история показывает готовность клиента исполнять кредитные обязательства. Во-вторых, потенциальный заемщик может не являться действующим клиентом банка, в связи с чем возрастает вероятность отсутствия его транзакционной истории.

Кредитная история заявителя, как и транзакционная представляет собой многомерный временной ряд. Каждый элемент ряда — это предыдущий кредит заемщика,

который описывается набором вещественных и категориальных признаков. При этом набор признаков может меняться, в зависимости от организации, в которой накапливаются данные кредитных историй.

Кредитный скоринг решает разнообразные задачи, включая оценку рисков в условиях экономических кризисов. Так, Albanesi и Vamossy (2024)[1] в своей работе исследуют прогнозирование дефолта в период глобального финансового кризиса 2008 – 2009 годов, вызванного коллапсом рынка недвижимости. Авторы проанализировали как клиенты выплачивают кредиты в конкретный кризисный период по сравнению с докризисным и посткризисным периодами.

Цель и задача исследования – разработать метод прогнозирования дефолта клиента банка при выдаче кредита.

Для достижения данной цели были поставлены следующие задачи:

1. Провести анализ и обработку данных.
2. Обучить модели классификации.
3. Подобрать гиперпараметры моделей, позволяющие выявлять ошибочно предсказанных дефолтных клиентов банка.
4. Проанализировать результаты очистки данных.

Объектом исследования является выданный кредит банком, а также наступление или отсутствие дефолта у клиента.

Предметом исследования является кредитный скоринг, т.е. определение риска невозврата кредита.

Для достижения целей и задач исследования использовались следующие методы:

- машинного обучения;
- статистического анализа;
- оценивания моделей;
- визуализации исторических данных.

Информационной базой исследования являлись платформы: kaggle, github, stackoverflow, научная литература .

Структура и объем работы. Курсовая работа состоит из 2 глав, заключения, списка литературы и 2 приложений.

Объем исследовательской работы: 23 страницы.

В данной курсовой работе используются следующие признаки из датасета:

[1]Albanesi S., Vamossy, D.F. Credit Scores: Performance and Equity // NBER Working Paper. 2024. No. 32917. C. 1–59.

- сумма кредита (LIMIT_BAL);
- пол (SEX);
- образование (EDUCATION);
- семейное положение (MARRIAGE);
- возраст (AGE);
- статистика отсутствия или наличия дефолта за 6 месяцев (PAY_0, PAY_2, PAY_3, PAY_4, PAY_5, PAY_6);
- счет за каждый месяц (BILL_AMT1, BILL_AMT2, BILL_AMT3, BILL_AMT4, BILL_AMT5, BILL_AMT6);
- опата счета за каждый месяц (PAY_AMT1, PAY_AMT2, PAY_AMT3, PAY_AMT4, PAY_AMT5, PAY_AMT6);
- статус кредита (DEFAULT).

Следует отметить, что для достижения точности алгоритмов:

a) не все признаки необходимы;

b) часть клиентов из датасета будут удалены.

Кроме этого датасет содержит признаки, которые в моделях МО не будут использоваться:

- пол;
- образование;
- возраст;
- семейное положение.

ГЛАВА 1. ОСНОВЫ КРЕДИТНОГО СКОРИНГА

1.1 Методы машинного обучения

1.1.1 KNN

Для применения алгоритмов машинного обучения необходимо тщательно подбирать методы, обеспечивающих высокую точность прогнозирования. В исследовании Markov et al. (2022)[1] анализируется популярность различных алгоритмов. Результаты показали, что алгоритм логистической регрессии занимает лидирующую позицию по частоте использования. Однако этот алгоритм демонстрирует хорошую точность, когда кластеры можно разделить гиперплоскостью, то есть данные имеют приближенно линейную структуру.

Алгоритм K-ближайших соседей (KNN) – один из наиболее простых и известных методов машинного обучения с учителем, относящийся к классу непараметрических моделей[2]. В реализации $KNeighborsClassifier$ рассматриваются три ключевых гиперпараметра: число K ближайших соседей, метрика расстояния и способ учета весов соседей. Обучение алгоритма осуществляется на основе имеющихся образцов и выполняется за счет следующих шагов:

1. определение количества K соседей;
2. выбор метрику расстояния;
3. определение K ближайших точек из обучающего набора данных, которые ближе всего к тестовой точке x;
4. классификация тестовой точки x на основе большинства голосов.

Параметр K определяет количество ближайших точек из обучающего набора данных для классификации нового объекта. При выборе небольших значений параметра K модель подстраивается под обучающие данные, что делает ее гибкой, однако это повышает вероятность риска переобучения. В свою очередь, большие значения K приведут к сглаживанию границ между классами, что может вызвать недообучение модели. Оптимальное значение K подбирается с помощью методов кросс-валидации.

Поскольку алгоритм KNN относится к метрическим алгоритмам, по умолчанию применяется метрика Минковского[3] для измерения расстояния между объектами. С ее

[1] Anton Markov, Zinaida Seleznyova, Victor Lapshin. Credit scoring methods: Latest trends and points to consider // The Journal of Finance and Data Science. 2022. №8. С. 180–201.

[2] KNeighborsClassifier [Электронный ресурс] / Python-библиотека для машинного обучения. URL: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html (дата обращения: 28.01.2025) Загл. с экрана.

[3] Герман Минковский (1864 – 1909 гг.) – немецкий математик и физик, разработавший геометрическую теорию чисел и геометрическую четырехмерную модель теории относительности.

помощью оценивается расстояние и определяется принадлежность клиента к соответствующему кластеру 0 или 1 на основе близости к объектам из обучающей выборки.

$$d(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}}$$

При $p = 1$ расстояние Минковского называется расстоянием Манхэттена, а при $p = 2$ - Евклидовым[4] расстоянием.

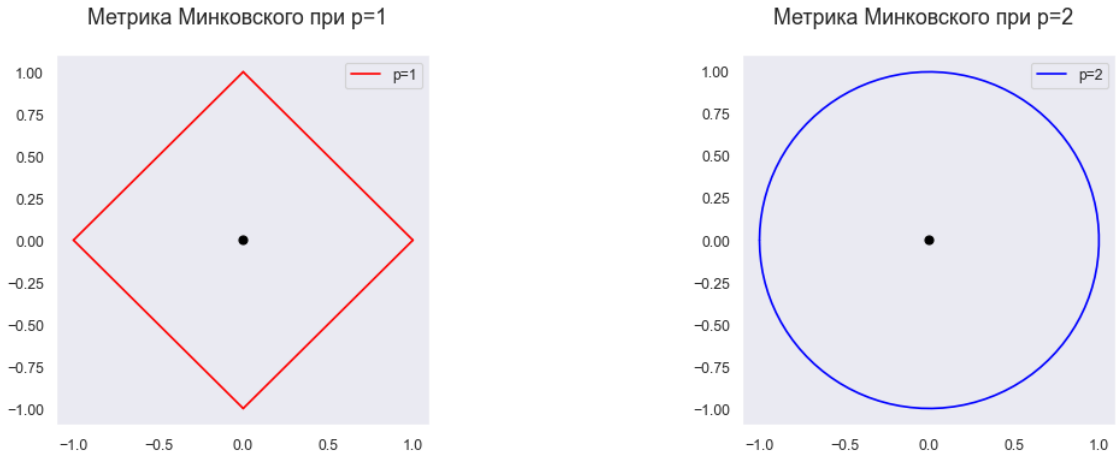


Рисунок 1 – Метрика Минковского при $p = 1$, $p = 2$

Источник: составлено автором на основе: Метрические методы [Электронный ресурс] / Яндекс Практикум. URL: https://education.yandex.ru/handbook/ml/article/metricheskiye-metody (дата обращения: 28.01.2025).

При наличии признаков, измеряемых в разных масштабах, перед использованием евклидовой метрики необходимо провести стандартизацию данных для того, чтобы признаки вносили одинаковый вклад в расчет расстояния между объектами.

Важным гиперпараметром является голосование или взвешивание условных вероятностей ($weights$), который задает степень влияния голоса каждого соседа на итоговую принадлежность к классу. Существует два подхода:

1. `«uniform»` – все K ближайших соседей имеют одинаковый вес при голосовании, и класс наблюдаемого объекта определяется большинством голосов; т.е., если среди K соседей преобладают объекты определенного класса, то наблюдение будет отнесено к этому классу, даже если некоторые некоторые соседи другого класса будут расположены ближе.

2. `«distance»` – веса соседей зависят от расстояния до наблюдаемого объекта: чем меньше расстояние, тем больше вес, что позволяет наиболее близким точкам оказывать большее влияние на итоговое решение. Исходя из этого, даже если объектов определенного класса будет в меньшем количестве среди ближайших соседей,

[4]Евклид (325 – 265 гг. до н.э.) – древнегреческий математик, автор одного из наиболее древних теоретических математических трактатов.

их более высокий вес будет восполнять числовое преимущество других классов, и тогда наблюдаемый объект будет относиться к этому классу.

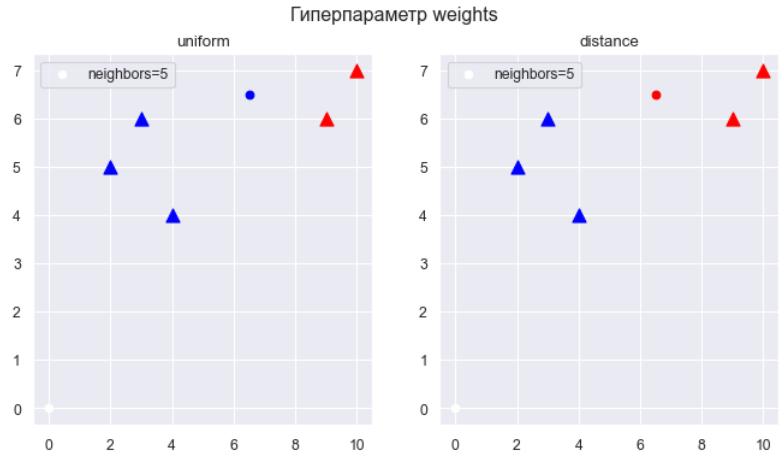


Рисунок 2 – Гиперпараметры – «uniform» , «distance»

Источник: составлено автором на основе: Метрические методы [Электронный ресурс] / Яндекс Практикум. URL: https://education.yandex.ru/handbook/ml/article/metricheskiye-metody (дата обращения: 28.01.2025).

Пример предсказания модели KNN на клиенте с индексом $M = 28000$ представлен в листинге 1 (см. Приложение А). Для выбранного клиента извлекаются данные об объеме кредита, выписках по счетам за последние 6 месяцев и платежная история в течение полугода. Матрица X состоит из 13 признаков (столбцов) и 30000 строк, где каждая строка – это кредитная история клиента в течение полугода и объем кредита, а столбец Y – это метки со значениями 0 или 1, предсказываемые алгоритмом.

Для данного клиента были получены предсказания меток с разными гиперпараметрами в листингах 2 и 3 (см. Приложение А):

1. При weights=«distance» наибольшее влияние оказывают соседи, относящиеся к классу клиентов без дефолта. Следовательно, модель предсказывает метку 1.

2. При weights=«uniform» вес ближайших соседей одинаковый. В нашем примере среди K ближайших соседей преобладают клиенты без дефолта, поэтому модель предсказывает метку 0, что является ошибочным предсказанием.

Исходя из приведенного примера видно, что выбор гиперпараметров «distance» и «uniform» оказывают существенное влияние на предсказательную способность алгоритма KNN. Следовательно, очень важно тщательно подбирать гиперпараметры, поскольку от их корректного подбора зависит точность модели и, соответственно, оценка риска невозврата кредита, который банк может выдать клиенту.

ГЛАВА 2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ АЛГОРИТМОВ И РЕЗУЛЬТАТЫ

2.1 Описание данных и процесс их подготовки

В данном исследовании используется датасет, содержащий информацию о просроченных платежах, кредитных данных, истории платежей и выписках по счетам клиентов кредитных карт на Тайване за период с апреля по сентябрь 2005 года[1]. Загрузка датасета представлена в листинге 4 (см. Приложение Б) .

Одной из ключевых задач кредитного скоринга является идентификация клиентов, у которых платежеспособность сомнительна, поскольку для таких клиентов модели МО могут сделать неверный прогноз, тем самым увеличивая риски для банка. В связи с этой проблемой возникает следующая идея по оптимизации рисков выдачи кредитов определенным клиентам.

В листинге 5 (см. Приложение Б) представлен алгоритм, удаляющий клиентов с дефолтом на основании анализа их платежной истории. Для начала создаются два дополнительных столбца: $removed_0$ – клиенты с просроченными платежами, и $removed_1$ – клиенты, осуществляющие своевременные платежи. Для каждого клиента проверяется платежная история: в случае, если все платежи больше нуля (т.е. просрочены), то клиент принадлежит к группе $removed_0$; если же все платежи меньше нуля (т.е. своевременные платежи), клиент относится к группе $removed_1$. Далее проводится фильтрация по метке $labels$, которая характеризует принадлежность клиента к одной из двух категорий: без дефолта ($label = 0$) или с дефолтом ($label = 1$). На основе данной фильтрации выделяются две группы клиентов:

I. клиенты, полностью погасившие кредит, но имеющие просроченные платежи по всем месяцам кредитной истории. Такие клиенты рассматриваются как дефолтные, т.к. счет по кредиту в установленный срок не оплачивается;

II. клиенты, не имевшие просрочек по всем месяцам кредитной истории, но не полностью погасившие кредит.

Таким образом, с помощью данного алгоритма можно выявить и удалить из выборки таких клиентов, которые могут негативно влиять на предсказательную способность модели МО за счет неправильной интерпретации платежной истории клиентов.

Далее проводится анализ различных групп удаленных клиентов, где группировку можно сделать по различным критериям. На рисунке 3 представлена группировка

[1]Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025). Загл. с экрана.

по возрастным категориям: синий столбец соответствует классу 0 ($label = 0$) – клиентам без дефолта, а оранжевый столбец соответствует классу 1 ($label = 1$) – клиенты с просроченными платежами. Первоначально были рассмотрены удаленные клиенты по возрастной категории. График показывает, что в возрастной категории от 40 до 50 лет преобладают клиенты с дефолтом, в то время как клиенты без дефолта преобладают в возрастной группе от 25 до 30 лет.

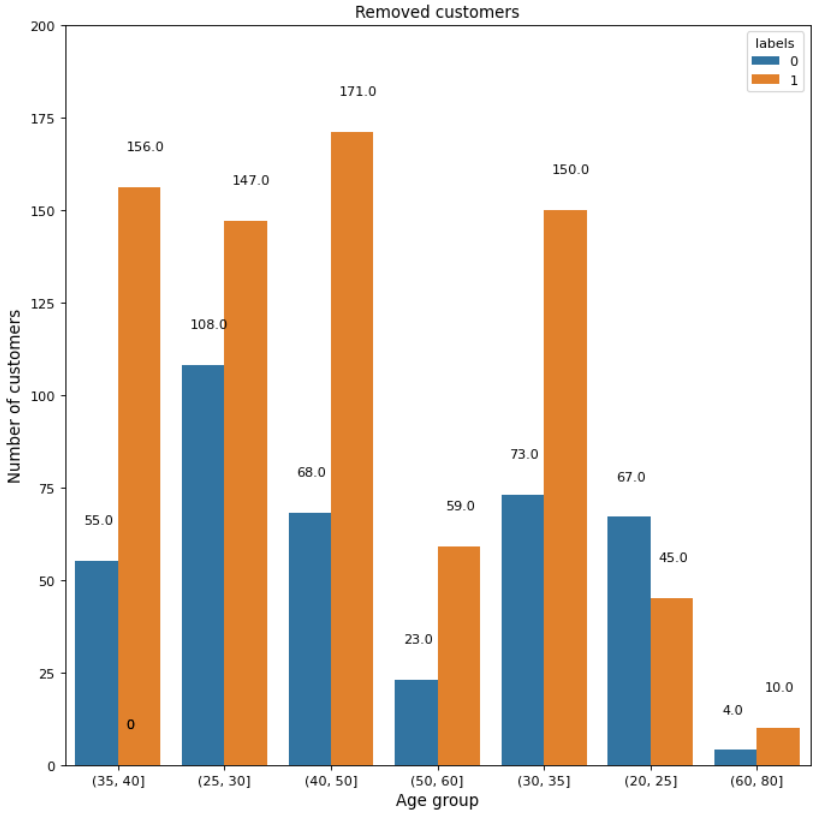


Рисунок 3 – Возрастная группа удаленных клиентов

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

Также проводится анализ статистики удаленных клиентов в зависимости от суммы выданного кредита. На рисунке 4 основная часть этих клиентов оформляет кредиты до 300000 тайваньских долларов, при этом большинство из удаленных клиентов имеют кредиты, не превышающие 50000 тайваньских долларов. Данная категория клиентов имеет наиболее стабильное финансовое положение.

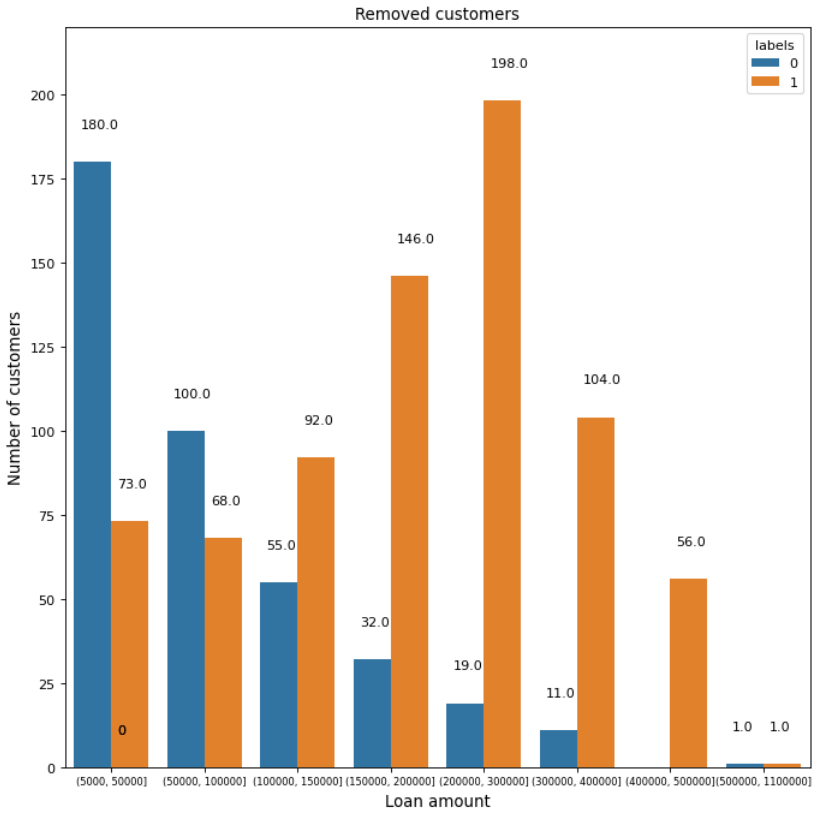


Рисунок 4 – Сумма выданного кредита среди удаленных клиентов

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

Из исходной выборки, состоящей из 30000 клиентов, было удалено 1136 клиентов. На данном обновленном наборе данных будут обучаться модели МО.

2.2 Реализация алгоритмов

Principal ComponentAnalysis (PCA) – метод сокращения размерности данных, который позволяет уменьшать количество признаков в наборе данных с сохранением максимального объема исходной информации[2]. Основная цель метода PCA заключается в нахождении новых, независимых друг от друга компонент, способных объяснять большую часть информации.

В листинге 6 (см. Приложение Б) осуществляется импорт класса PCA из библиотеки scikit-learn для проведения анализа главных компонент. Из исходного $DataFrame$ извлекаются такие признаки, как сумма выданного кредита ($LIMIT_BAL$), задолженность за последние 6 месяцев ($BILL_AMT1$ – $BILL_AMT6$) и суммы платежей (PAY_AMT1 – PAY_AMT6), которые сохраняются в переменной X. Дополнительно

[2]PCA [Электронный ресурс] / Python-библиотека для машинного обучения. URL: https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html(дата обращения: 28.01.2025) Загл. с экрана.

определяется новая переменная Y, содержащая метки классов ($labels = 0$ или $labels = 1$).

После подготовки данных применяется метод PCA для обучения на выбранном наборе признаков. С помощью метода $pca.explained_variance_ratio_$ определяется вклад каждой компоненты в общую информативность. Метод показал, что $LIMIT_BAL$, $BILL_AMT1$ – $BILL_AMT6$, PAY_AMT1 – PAY_AMT6 вносят существенный вклад в информативность данных о клиентах.

2.2.1 Реализация алгоритма KNN

Перед настройкой гиперпараметров модели KNN с использованием $GridSearch$ необходимо создать базовую модель без оптимальных гиперпараметров, которая позволит оценить исходную точность алгоритма на текущих данных. Алгоритм $K-NearestNeighbors$ (KNN) представляет собой метод классификации, который определяет класс нового объекта на основе анализа ближайших соседей в обучающей выборке данных. Алгоритм KNN с параметрами по умолчанию реализован в листинге 7 (см. Приложение Б).

Из библиотеки $sklearn$ импортируются функции: $train_test_split$ для разбиения данных на обучающую и тестовую выборки, $accuracy_score$ для оценки точности предсказания модели, а также класс $KNeighborsClassifier$ для реализации метода k-ближайших соседей. Функция $train_test_split$ разделяет данные так, что 80% выборки используется для обучения модели, а 20% – для тестовой выборки. Параметр $random_state = 0$ обеспечивает неизменность выборок при каждом запуске кода. После разделения данных создается базовая модель KNN с параметрами по умолчанию, что позволит сравнить текущую работу модели до и после оптимизации гиперпараметров. Пример реализован в листинге 8 (см. Приложение Б).

Данная модель обучается на полном наборе данных X и Y. Для предотвращения эффектов переобучения или недообучения модели, необходимо подобрать оптимальные гиперпараметры алгоритма KNN. Метод $GridSearchCV$[3] применяется для автоматического подбора гиперпараметров, поскольку их оптимальное значение существенно влияет на качество предсказаний модели. Исходя из заданного множества значений гиперпараметров, GridSearchCV перебирает все возможные комбинации гиперпараметров и с помощью алгоритма кросс-валидации выбирает наилучший набор параметров. Реализация метода $GridSearchCV$ для KNN представлен в листинге 9 (см. Приложение Б):

Комментарий к листингу 9:

а) $parameter_space$ – словарь искомых параметров;

[3]GridSearchCV [Электронный ресурс] / Python-библиотека для машинного обу-чения. URL: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html(дата обращения: 28.01.2025). Загл. с экрана.

b) $grid_clf$ – модель gridsearch;

c) $grid_clf.fit(X_train, Y_train.travel())$ – тренировка модели gridsearch на тренировачных данных;

d) $grid_clf.best_estimator_$ – массив оптимальных гиперпараметров.

2.2.2 Точность KNN на оптимальных гиперпараметрах

Для оценки точности работы модели KNN используется функция $accuracy_score$, которая вычисляет долю правильно классифицированых объектов на основе общего числа наблюдений. Точность модели составила 0.99342, что указывает на высокую эффективность модели. Пример кода для оценки точности доступен в листинге 10 приложении Б.

Приведенный код реализует предсказания меток всего набора данных и сохраняет фактические метки. Затем с использованием метода $score()$ вычисляется точность модели на тренировочных и тестовых выборках, что позволяет оценить качество обучения модели. Тем не менее, данный метод показывает только общую точность предсказаний, что может быть недостаточным критерием оценки качества модели. При дисбалансе классов, когда большинство клиентов в банке относятся к заемщикам без дефолта, модель может показать высокую точность, предсказывая всех, как надежных клиентов. В исследовании Markov et al. (2022) отмечается, что более 50% исследователей в качестве метрики точности предпочитают использовать $confusion\ matrix$ (матрицу ошибок)[4]. Данная матрица позволяет визуализировать результаты классификации клиентов по 4 группам:

I. $TruePositives$ (TPs) – объекты, фактически принадлежавшие положительному классу были правильно определены моделью.

II. $FalseNegatives$ (FNs) – объекты, относящиеся к положительному классу, модель ошибочно относит их к отрицательному.

III. $FalsePositives$ (FPs) – объекты, принадлежавшие отрицательному классу, ошибочно предсказываются как положительные.

IV. $TrueNegatives$ (TNs) – объекты, правильно классифицированные, как отрицательные.

[4]Markov et al. (2022). С. 180–201.

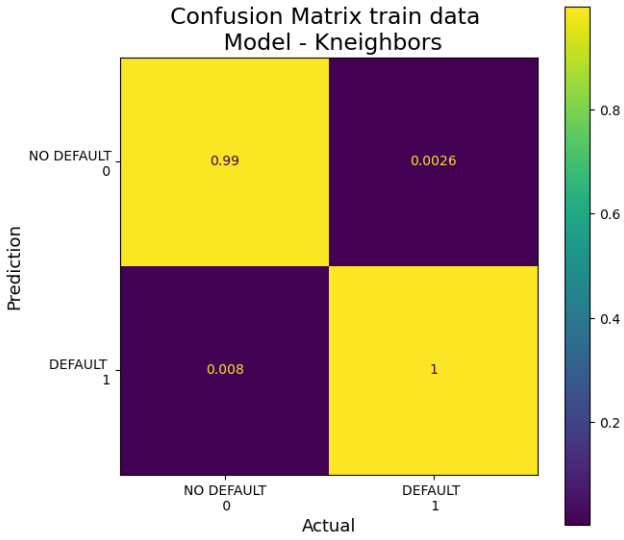


Рисунок 5 – Confusion matrix для тренировочных данных

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

В представленной матрице отражается точность алгоритма KNN на тренировочных данных (см. Рисунок 5). Матрица показывает, что модель практически не допускает ошибок при классификации: значение FN составляет 0.008, что указывает на низкую вероятность ошибочного предсказания дефолтного заемщика к классу заемщиков без дефолта. $FP = 0.0026$ показывает низкую вероятность предсказаний заемщиков без дефолта как дефолтных. Кроме того модель демонстрирует почти идеальную точность на обучающих данных, где $TP = 0.99$ и $TN = 1$. Это означает, что клиенты с метками 0 и 1 были правильно предсказаны. Код представлен в листинге 11 (см. приложение Б).

После этого была построена $confusion\ matrix$ для оценки точности алгоритма KNN на тестовых данных (см. Рисунок 6). Матрица показывает, что 99% клиентов без дефолта и 100% клиентов с дефолтом были правильно классифицированы моделью, что свидетельствует о низкой вероятности ошибок при предсказании. Несмотря на это, 0.43% клиентов с дефолтом были ошибочно отнесены к классу заемщиков без дефолта и 0.72% клиентов без дефолта – предсказаны как дефолтные. По сравнению с матрицей ошибок на обучающих данных FP и FN немного увеличились в тестовой выборке, что указывает на незначительную степень переобучения. При этом подобранные гиперпараметры с помощью $GridSearch$ дают высокую точность. Код к рисунку 6 представлен в листинге 12 (см. приложение Б).

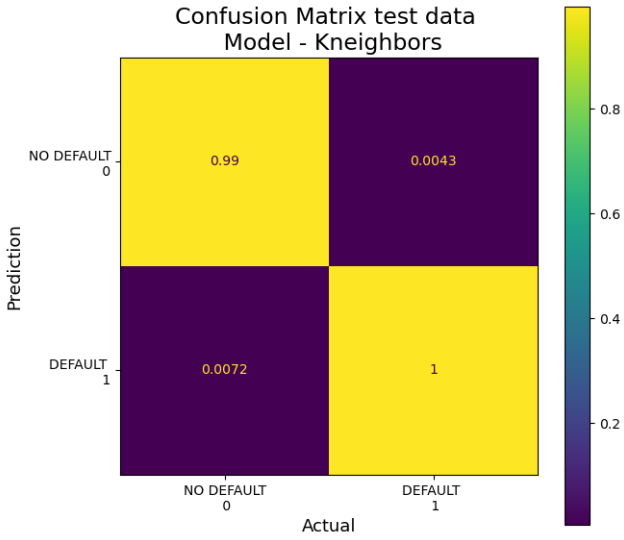


Рисунок 6 – Матрица ошибок для тестовых данных

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

На последнем этапе составлена *confusion matrix*, охватывающая весь набор данных (см. Рисунок 7). Результаты показывают, что 99% клиентов без дефолта и 100% клиентов с дефолтом были верно распознаны. Тем не менее наблюдается небольшая вероятность отнесения надежных заемщиков к ненадежным и наоборот. Результат демонстрирует высокую точность модели как на тренировочных, так и на тестовых данных. Код реализован в листинге 13 (см. приложение Б).

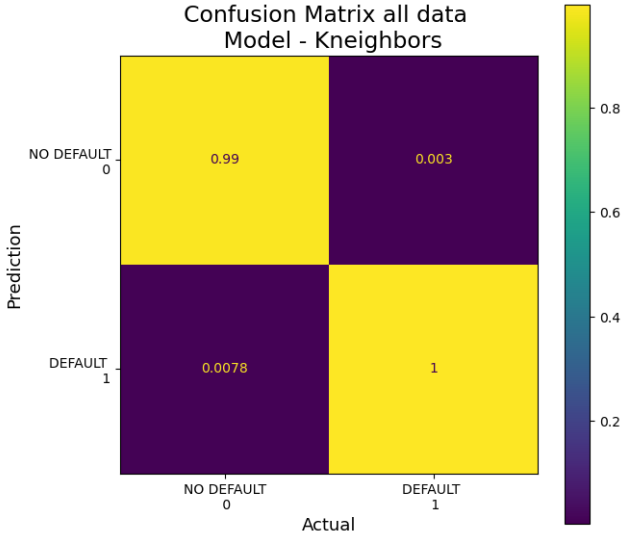


Рисунок 7 – Матрица ошибок для всех данных

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

Для более точного оценивания качества работы модели в каждом классе по отдельности применяются дополнительные метрики точности, как *precision* (точность), *recall* (полнота) и $f1-score$ (f-мера). *Precision* показывает долю объектов, предсказанных классификатором положительными, которые действительно относятся к положительному классу. В то время как *recall* означает какая доля объектов положительного класса была верно выявлена из общего количества положительного класса. Следовательно *precision* отвечает за способность различать классы, а *recall* показывает способность модели находить объекты определенного класса. f-мера это среднее гармоническое *precision* и *recall*, которая показывает единую оценку модели и вычисляется по следующей формуле:

$$f_\beta = (1 + \beta^2) \cdot \frac{precision \cdot recall}{\beta^2 \cdot precision + recall} \tag{1}$$

Значение β определяет вес точности в метрике:

- при $\beta = 1$ метрика учитывает precision и recall в равной степени, достигает максимального значения и превращается в $f1 - score$;
- при $\beta > 1$, то большее значение придается полноте ($recall$);
- при $\beta < 1$, то большее внимание уделяется точности ($precision$).

На основе тренировочных данных (см. Таблицу 1) были вычислены метрики точности. Для класса 0 (клиенты без дефолта) модель показывает высокую точность ($precision = 0.99$) и $recall = 1.00$, указывая на правильную классификацию объектов. Для класса 1 (клиенты с дефолтом) значения метрик остаются высокими – *precision* составляет 1.00, а $recall = 0.97$, что демонстрирует точное распознавание объектов. Высокие показатели $f1-score$ показывают хорошую сбалансированность оценок обоих классов. Показатель support указывает, что в тренировочной выборке 18380 объектов относятся к классу 0, а 4711 к классу 1.

Таблица 1 – Метрики точности модели на тренировочных данных

label	precision	recall	f1-score	support
0	0.99	1.00	1.00	18380
1	1.00	0.97	0.98	4711

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

Аналогичные расчеты были проведены для тестовых данных (см. Таблицу 2). Классу 0 соответствуют значения $precision = 0.99$, $recall = 1.00$, $f1 - score = 1.00$, что говорит о высокой точности предсказания данного класса. В классе 1 $precision = 1.00$, $recall = 0.97$, $f1 - score = 0.98$. Несмотря на то, что модель практически не допускает

ошибок при определении клиентов с дефолтом, значение $recall < 1$ говорит о возможности пропусков некоторых объектов. В тестовой выборке 4586 клиентов принадлежат к классу 0, 1187 – к классу 1.

Таблица 2 – Метрики точности модели на тестовых данных

label	precision	recall	f1-score	support
0	0.99	1.00	1.00	4586
1	1.00	0.97	0.98	1787

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

После этого были рассчитаны метрики точности для полного набора данных (см. Таблицу 3). Результат остается на таком же высоком уровне для обоих классов.

Таблица 3 – Метрики точности модели на всех данных

label	precision	recall	f1-score	support
0	0.99	1.00	1.00	22966
1	1.00	0.97	0.98	5898

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

Для оценки качества модели KNN на сомнительных клиентах была построена *confusion matrix* (см. Рисунок 8). Это позволило рассмотреть насколько правильно модель определяет их метку (0, 1).

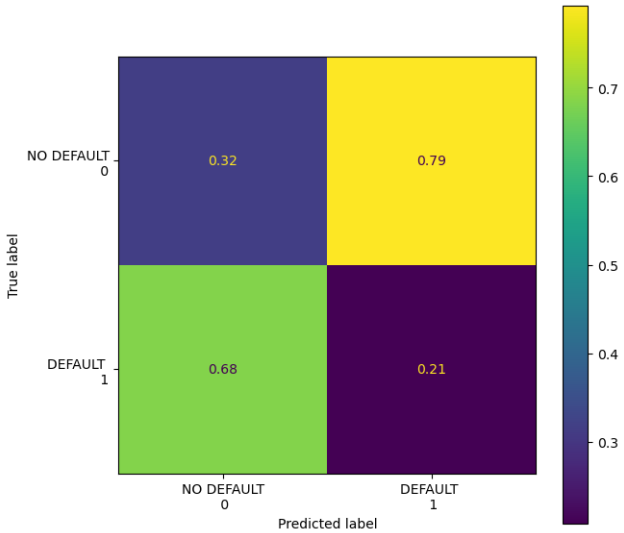


Рисунок 8 – Confusion matrix для удаленных клиентов

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

Высокие значения $FalseNegatives$(0.68) и $FalsePositives$(0.79) показывают, что модель часто ошибается, относя дефолтных клиентов к группе клиентов без дефолта

и наоборот. *Confusion matrix* (см. Рисунок 8) подтверждает необходимость удаления клиентов с сомнительной кредитной историей, т.к. модель допускает большое количество ошибочных предсказаний. Код представлен в листинге 14 (см. приложение Б).

Модель *KNN* реализует предсказание меток *y_pred* удаленных клиентов, где *y_true* - истинные метки из датасета. *Confusion matrix* (см. Рисунок 8) показывает точность предсказанных меток удаленных клиентов.

Значения в таблице 4 показывают, что модель плохо предсказывает клиентов с дефолтом (метка 1). *Precision* составляет 0.21, а *recall* = 0.02, означающее, что модель практически не способна идентифицировать клиентов с меткой 1. В то же время для класса без дефолта (метка 0) *precision* = 0.32, а *recall* = 0.84, что означает высокую вероятность ложных предсказаний. Для обоих классов низкие значения $f1-score$ подтверждают неэффективность работы модели на этих данных.

Таблица 4 – Метрики точности модели на удаленных клиентах

label	precision	recall	f1-score	support
0	0.32	0.84	0.46	398
1	0.21	0.02	0.04	738

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

2.3 Деревья решений

Decision Tree (дерево решений) – это непараметрический алгоритм обучения с учителем, используемый для задач классификации и регрессии[5]. Он имеет иерархическую структуру в виде дерева, состоящую из корневого узла, ветвей, внутренних и листовых узлов. В задаче кредитного скоринга используется *classification tree* (дерево классификации), где каждое наблюдение принадлежит к наиболее часто встречающемуся классу обучающих данных. Тем не менее следует учитывать не только сам результат терминального узла, но и понимать насколько уверенно алгоритм относит заемщика к классу надежных или ненадежных клиентов. По умолчанию в *DecisionTreeClassifier* применяется индекс Джини [6], который измеряет общий разброс классов в узле дерева.

$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}),$$

где $\hat{p}_{mk}$ - доля обучающих наблюдений в m-й области, принадлежащих k-му классу.

Если $\hat{p}_{mk}$ близки к 0 или 1, т.е. в узле доминирует один класс, то индекс Джини принимает небольшие значения. Этот показатель показывает степень неоднородности

[5]DecisionTreeClassifier [Электронный ресурс] / Python-библиотека для машинного обучения. URL: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html(дата обращения: 28.01.2025). Загл. с экрана.

[6]Коррадо Джини (1884 – 1965 гг.) – итальянский статистик, социолог, демограф и экономист.

в узле: чем выше значение индекса Джини, тем более разнородный узел, и наоборот.

Альтернативой индексу Джини является энтропия, которая также оценивает разнообразие классов в узле. Если $\hat{p}_{mk}$ близка к 0 или 1, то энтропия принимает малое значение.

$$D = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$$

Оба критерия показывают насколько однородны объекты в узле. Индекс Джини и энтропия работают схожим образом, однако индекс Джини более чувствителен к разбиению узла.

2.3.1 Реализация алгоритма

В этом разделе описана реализация алгоритма *Decision Tree*, включающая подготовку, фильтрацию и обучение на данных. В листинге 15 (см. Приложение Б) формируется матрица X_tree со столбцами: $LIMIT_BAL$, $PAY_0 - PAY_6$. Одновременно с этим из выборки исключаются «сомнительные» клиенты и создается целевой вектор, указывающий является ли клиент недефолтным или дефолтным. После фильтрации данных происходит разбиение данных на 2 выборки: тренировочную (80%) и тестовую (20%). На следующем этапе создается и обучается модель *Decision Tree* со следующими гиперпараметрами:

a) $max_depth = 10$ – ограничивает глубину дерева до 10 уровней для предотвращения переобучения;

b) $min_samples_leaf = 2$ – указывает минимальное количество объектов в терминальном узле для предотвращения больших разветвлений, тем самым делая дерево более устойчивым;

c) $random_state = 42$ – фиксирует случайность, чтобы при каждом запуске построение дерева было одинаковым.

После обучения оценивается точность модели с помощью метода $score()$, который показывает насколько хорошо модель классифицирует клиентов.

Точность модели *DecisionTreeClassifier* равна 0.86169. Это означает, что 86,17% предсказаний совпадают с фактическими метками объектов. В таблице 5 указаны метрики точности $DecisionTreeClassifier$.

Таблица 5 – Метрики точности на всех данных Decision Tree

label	precision	recall	f1-score	support
0	0.87	0.96	0.92	22966
1	0.77	0.46	0.58	5898

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

Из таблицы 5 видно, что для класса 0 (клиенты без дефолта) модель показывает следующие результаты:

a) $precision = 0.87$ – показывает, что 87% клиентов, относящихся к классу 0, на самом деле не просрочили начисления по кредиту;

b) $recall = 0.96$ – означает, что модель правильно обнаруживает 96% недефолтных клиентов;

c) $f1 - score = 0.92$ – среднее между $precision$ и $recall$ показывает хорошую сбалансированность, т.е. модель достаточно распознает хороших заемщиков.

Однако, для меток 1 (клиенты с дефолтом) точность всех трех метрик несколько ниже, чем для меток 0 (клиенты без дефолта).

a) $precision = 0.77$ – показывает, что только 77% дефолтных клиентов были действительно правильно предсказаны;

b) $recall = 0.46$ – среди всех заемщиков с дефолтом, модель верно распознала только 46%;

c) $f1 - score = 0.58$ – среднее между $precision$ и $recall$ показывает низкую сбалансированность и означает, что модель плохо справляется с классификацией клиентов с меткой 1.

В ходе исследования выяснилось, что модель неверно классифицирует дефолтных заемщиков, относя их к классу 0, несмотря на наличие просроченных платежей. Подобная ситуация объясняется тем, что такие клиенты продолжают производить платежи, и по этой причине модель относит их к классу недефолтных заемщиков. Для понимания этой проблемы данные были визуализированы. Так как метрики точности для класса 0 достаточно высоки, основное внимание уделяется классу дефолтных клиентов. В частности, исследуется история просрочек по месяцам PAY_0 – PAY_6. Графическая визуализация этих столбцов представлена ниже (см. Рисунок 9).

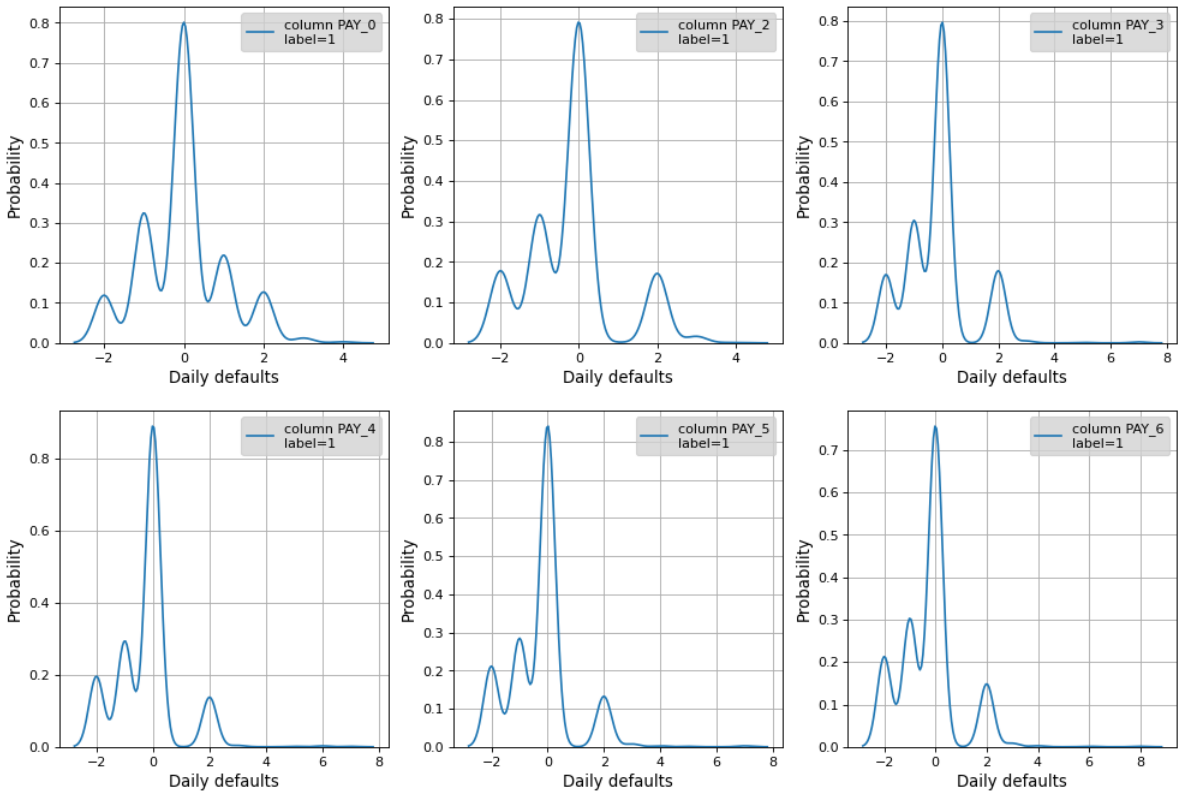


Рисунок 9 – Визуализация клиентов с наличием дефолта

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

Графический анализ показывает, что среди дефолтных клиентов значительная часть своевременно оплачивала счета по кредитам по некоторым месяцам, при этом имея задолженность по оплате счета за конкретный месяц. Соответственно это привело к тому, что модель неверно классифицировала их как платежеспособных. Для улучшения метрик точности были удалены дефолтные клиенты, которые имеют значение 0 в столбце PAY_0.

В листинге 16 (см. Приложение Б) X_pay преобразуется в массив, из которого, начиная со второго столбца, выделяются столбцы PAY_0 - PAY_6. После этого в $DataFrame$ добавляется новый столбец $removed_0$, который содержит объекты, не помеченные на удаление. Если клиент удовлетворяет двум условиям (в листинге эти критерии указаны в строке 6), то он обозначается $removed_0 = True$ для исключения из обучающей выборки:

$$\begin{aligned} &a)\ PAY_0 = 0, \\ &b)\ y = 1. \end{aligned} \tag{1}$$

Первое условие в (1) означает, что мы выбираем столбец PAY_0 и рассматриваем только тех клиентов, которые оплатили выставленный счет по кредиту вовре-

мя (это соответствует значению 0). Второе условие означает, что клиент должен быть дефолтным (метка 1). В итоге происходит удаление клиента, который является дефолтным, но в первый месяц оплачивал начисление по кредиту пунктуально. Столбец PAY_0 выбран не случайно, т.к. большинство клиентов стараются в первый месяц оплачивать начисления по кредиту вовремя. По этой причине в столбце PAY_0 скопилось большое количество дефолтных клиентов. Была выдвинута гипотеза, что, если удалить дефолтных клиентов, соответствующих критерию (1), то метрики алгоритма $DecisionTreeClassifier$ для дефолтных клиентов улучшатся. С математической точки зрения необходимо реализовать тест на равенство и неравенство двух долей (основная и альтернативная гипотезы):

$$\mathcal{H}_0 : \ p_0 = p_i, \ i = 2, \ldots, 6$$

$$\mathcal{H}_1 : \ p_0 > p_i, \ i = 2, \ldots, 6,$$

где p_0, p_i - доли клиентов, удовлетворяющих условию (1) для сооответствующих столбцов. Смысл этого теста заключается в том, чтобы выявить сходства или различия клиентов согласно критерию (1). Психология заемщиков заключается в том, что в первый месяц почти все заемщики выплачивают счет по кредиту своевременно, и по этой причине алгоритм плохо классифицирует клиентов. После удаления клиентов, удовлетворяющих условиям (1), метрики точности показали следующие результаты:

Таблица 6 – Метрики точности после удаления аномалий

label	**precision**	**recall**	**f1-score**	**support**
0	0.91	0.95	0.93	23364
1	0.69	0.56	0.62	4748

Источник: составлено автором на основе: Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv (дата обращения: 28.01.2025).

Можно заметить, что для дефолтных клиентов (метки 1) *recall* увеличился до 0.56, т.е. теперь модель распознает 56% дефолтных клиентов. Стоит отметить, что *precision* снизился до 0.69 и показывает больше ложных предсказаний, когда ошибочно относит добросовестного клиента к классу 1. Тем не менее метрика $f1 - score$ увеличилась до 0.62, что показывает тенденцию к выравниванию метрик *recall* и *precision*. Таким образом, удаление клиентов по критерию, описанному выше, приводит к большей точности распознавания дефолтных клиентов. В рамках дипломной работы данная проблема будет более детально рассмотрена.

Заключение

В ходе исследования была рассмотрена задача кредитного скоринга на основе исторических данных, цель которой состояла в разработке и анализе моделей классификации, а также в оценке точности их предсказаний. В работе были проведены этапы от предварительной обработки данных, удаления клиентов с нестабильным финансовым поведением и до интерпретации их результатов. Особое внимание было направлено удалению аномалий, например, устранение дефолтных клиентов с отсутствием просрочек. Данное исключение позволило повысить общую точность моделей.

Для поставленной задачи были применены алгоритмы KNN и $Decision\,Tree$, производительность которых оценивалось по следующим ключевым метрикам: точность, $accuracy$, $precision$ (точность), $recall$ (полнота) и $f1-score$ (f-мера). Также, для поиска оптимальных гиперпараметров был применен метод $GridSearchCV$. Согласно результатам, несмотря на высокую общую точность, модели недостаточно хорошо распознают дефолтных клиентов. Анализ показал, что клиенты с нетипичными и противоречивыми характеристиками оказывают значительное влияние на работу моделей. Исключение таких клиентов позволило улучшить метрики качества моделей, а именно повысилась чувствительность к дефолтам, сократилось число ложных предсказаний и улучшилась надежность окончательных результатов.

Таким образом, исследование показывает, что модели МО являются эффективными инструментами кредитного скоринга, однако их результаты напрямую зависят от качества данных. Внедрение этих алгоритмов на практике демонстрирует их актуальность для банковской сферы.

СПИСОК ЛИТЕРАТУРЫ

Книги:

1. An Introduction to Statistical Learning with Applications in Python / J. G. James, D. Witten, T. Hastie., и др. Cham: Springer, 2023. С. 617. URL: https://bayanbox.ir/view/1060725898744657072/An-Introduction-to-Statistical-Learning-with-Applications-in-Python.pdf (дата обращения: 28.01.2025).

2. Python для сложных задач: наука о данных и машинное обучение / Плас Дж. Вандер : пер. с англ. – СПб. : ООО «Питер», 2018. С. 576. URL: https://iro23.ru/sites/default/files/plas_dzh._vander_-_python_dlya_slozhnyh_zadach_nauka_o_dannyh_i_mashinnoe_obuchenie_bestsellery_oreilly_-_2018.pdf (дата обращения: 28.01.2025).

3. Python и машинное обучение: машинное и глубокое обучение с использованием Python, scikit-learn и TensorFlow 2 / С. Рашка, В. Мирджалили : под редакцией Ю. Н. Артеменко. 3-е изд., пер. с англ. – СПб. : ООО «Диалектика», 2020. С. 848. URL: https://coollib.net/b/674877-sebastyan-rashka-python-i-mashinnoe-obuchenie (дата обращения: 28.01.2025).

Статьи:

4. Albanesi, S. Credit Scores: Performance and Equity / S. Albanesi, D.F. Vamossy, // NBER Working Paper. – 2024. – No. 32917. С. 1 – 59.

5. Markov, A. Credit scoring methods: Latest trends and points to consider / A. Markov, Z. Seleznyova, V. Lapshin // The Journal of Finance and Data Science. – 2022. – №8. С. 180 – 201.

6. Provenzano A. R. Machine learning approach for credit scoring / A. R. Provenzano, D. Trifirò, A. Datteo, L. Giada, N. Jean, A. Riciputi, et al. arXiv:2008.01687. – 2020. – С. 1 – 28.

Интернет-ресурсы:

7. Метрические методы [Электронный ресурс] / Яндекс Практикум. URL: https://education.yandex.ru/handbook/ml/article/metricheskiye-metody. — Загл. с экрана.

8. DecisionTreeClassifier [Электронный ресурс] / Python-библиотека для машинного обучения. – URL: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html. — Загл. с экрана.

9. Default of Credit Card Clients Dataset [Электронный ресурс] / Бизнес и конкурсная платформа по исследованию данных «Kaggle». – URL: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset/data?select=UCI_Credit_Card.csv. — Загл. с экрана.

10. GridSearchCV [Электронный ресурс] / Python-библиотека для машинного обучения. – URL: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html. — Загл. с экрана.

11. KNeighborsClassifier [Электронный ресурс] / Python-библиотека для машинного обучения. – URL: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html. — Загл. с экрана.

12. PCA [Электронный ресурс] / Python-библиотека для машинного обучения. – URL: https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html. — Загл. с экрана.

13. What is the k-nearest neighbors (KNN) algorithm? [Электронный ресурс] / International Business Machines. – URL: https://www.ibm.com/think/topics/knnl. — Загл. с экрана.

Приложение А
Алгоритм KNN и настройка гиперпараметров

Листинг 1 – Пример кода для предсказания метки клиента

```
M = 28000                                          # Client number
print('CREDIT', X.values[M][0], 'status', Y[M]) # Client's credit
    limit and status
print('BILL', X.values[M][1:7], 'status', Y[M]) # History of the
    client's credit accounts for the last 6 months
print('PAY ', X.values[M][7:],  'status', Y[M]) # Customer payment
    history for the last 6 months
knn.predict([X.values[M]])                         # Using KNN to
    predict client status
```

Листинг 2 – Пример кода с гиперпараметром «distance»

```
CREDIT 140000.0 status 1
BILL [17962. 17381. 18104. 17516. 18721. 18280.] status 1
PAY  [   0. 1300.    0. 1500.    0. 1900.] status 1
array([1])
```

Листинг 3 – Пример кода с гиперпараметром «uniform»

```
CREDIT 140000.0 status 1
BILL [17962. 17381. 18104. 17516. 18721. 18280.] status 1
PAY  [   0. 1300.    0. 1500.    0. 1900.] status 1
array([0])
```

Приложение Б
Обработка данных и оценка моделей

Листинг 4 – Загрузка датасета

```
df = pd.read_csv('UCI_Credit_Card.csv') # Download dataset from CSV file
df_copy = df.copy()                     # Creating a copy of the original DataFrame
df.rename(columns={"default.payment.next.month": "labels"}, inplace = True) # Rename the default.payment.next.month column as labels
df_copy.rename(columns={"default.payment.next.month": "labels"}, inplace = True) # Rename the same column to labels in the dataframe copy
```

Листинг 5 – Реализация бизнес-идеи

```
# Select payment history columns for the last 6 months
X_pay = df_copy.loc[:,['PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6']]
X_pay = np.array( X_pay )     # Convert to NumPy array
index = []
df_copy['removed_0'] = False # Creating a column for customers with overdue payments
df_copy['removed_1'] = False # Creating a column for customers with payments on time
for i,x in enumerate( X_pay):
    if min(x) > 0:            # Customer's payments were overdue
        index.append(i)
        df_copy.loc[i, ['removed_0']] = True
    if max(x) < 0:
        index.append(i)      # Customer's payments were on time
        df_copy.loc[i, ['removed_1']] = True
in_new_0 = np.logical_and( df_copy['removed_0'].values == True, df_copy['labels'].values == 0  ) # Index filtration of customers with overdue payments without default (label = 0)
in_new_1 = np.logical_and( df_copy['removed_1'].values == True, df_copy['labels'].values == 1  ) # Index filtration of customers with payments on time with default (label = 1)
in_new_0_1 = np.logical_or( in_new_0, in_new_1) # Combine two requirements
df = df.loc[~in_new_0_1,:]                      # Deleting these customers
```

Листинг 6 – Сокращение размерности с помощью PCA

```
from sklearn.decomposition import PCA       # Import PCA
X = df.loc[:, [ 'LIMIT_BAL','BILL_AMT1', 'BILL_AMT2',
          'BILL_AMT3', 'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6','
             PAY_AMT1',  'PAY_AMT2', 'PAY_AMT3','PAY_AMT4','PAY_AMT5'
             ,'PAY_AMT6',
]]                                          # Select credit limit,
   debt, payments to analyze PCA
Y = df['labels'].values                     # Create labels
pca = PCA()                                 # Create an object of
   class PCA
pca.fit( X )                                # Training of PCA
100*pca.explained_variance_ratio_.round(8) # Result significance of
   selected columns
```

Листинг 7 – Базовая модель KNN

```
from sklearn.model_selection import train_test_split # Import of the
   function of splitting data into training and test samples
from sklearn.metrics import accuracy_score          # Import metric
   for accuracy score
from sklearn.neighbors import KNeighborsClassifier   # Import model
   KNN
X_train, X_test, Y_train, Y_test = train_test_split( X, Y, test_size
   = 0.2, random_state = 0  ) # Splitting the data into training and
   test samples (80% and 20%)
knn = KNeighborsClassifier( )                       # Create a KNN
   model with default parameters
knn.fit( X, Y )                                     # Model training
```

Листинг 8 – Модель KNN с заданными гиперпараметрами

```
knn = KNeighborsClassifier(n_neighbors = 5, weights = 'uniform', p =
   2, metric = 'mincowski' )
```

Листинг 9 – Выбор гиперпараметров с помощью GridSearchCV

```
from sklearn.model_selection import GridSearchCV # Import method for
    hyperparameter selection
parameter_space = {
    'n_neighbors': [ 7, 9, 11, 21, 31, 51, 101 ], # Number of nearest
        neighbors
    'p':[1,2],                                    # Minkowski's Metric
    'weights' : [ 'uniform', 'distance' ]          # Weighing
       conditional probabilities
    }
grid_clf = GridSearchCV( knn, parameter_space )    # Parameter
    selection
grid_clf.fit( X_train, Y_train.ravel() )          # Model training
    with hyperparameter selection
print( 'grid_clf', grid_clf.best_estimator_ )      # Output of the
    best found parameters
```

Листинг 10 – Оценка точности модели

```
from sklearn.metrics import accuracy_score # Import accuracy score
    function
y_pred = knn.predict( X )                  # Predict labels for the
    entire dataset
y_true = Y                                 # Define the true labels
knn.score( X_train, Y_train )              # Model accuracy on the
    training data
knn.score( X_test, Y_test )                # model accuracy on the
    test data
```

Листинг 11 – Код к confusion matrix для тренировочных данных

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
                # Import confusion matrix function
y_pred_train = knn.predict(X_train)
                                                    # Predict labels for
    training data
cm = confusion_matrix(Y_train, y_pred_train, labels=[0, 1],
    normalize = 'pred') # Confusion matrix with normalization
print( str(cm) )
                                                                      #
    Print values of confusion matrix
disp = ConfusionMatrixDisplay( confusion_matrix = cm,
                               # Vizualization for confusion matrix
                                display_labels = ['NO DEFAULT \n 0','
                                   DEFAULT \n 1']
                              )
fig, ax = plt.subplots(figsize=(7,7),dpi = 100)
                                     # Set figure size and resolution
disp.plot(ax=ax)
                                                                      #
    Plot confusion matrix
plt.ylabel('Prediction',fontsize = 13)
                                                 # Label for the y axis
plt.xlabel('Actual',fontsize = 13)
                                                      # Label for the x
    axis
plt.title('Confusion Matrix train data \n Model - Kneighbors',
    fontsize = 17)     # Set the title
```

Листинг 12 – Код к confusion matrix для тестовых данных

```
y_pred_test = knn.predict(X_test)
                                                    # Predict labels for
    test data
cm = confusion_matrix(Y_test, y_pred_test, labels=[0, 1],  normalize
   = 'pred')  # Confusion matrix with normalization
print( str(cm) )
                                                                    #
   Print values of confusion matrix
disp = ConfusionMatrixDisplay( confusion_matrix = cm,
                              # Vizualization for confusion matrix
                                display_labels = ['NO DEFAULT\n 0','
                                   DEFAULT \n 1']
                              )
fig, ax = plt.subplots(figsize=(7,7),dpi = 100)
                                     # Set figure size and resolution
disp.plot(ax=ax)
                                                                    #
   Plot confusion matrix
plt.ylabel('Prediction',fontsize = 13)
                                                 # Label for the y axis
plt.xlabel('Actual',fontsize = 13)
                                                     # Label for the x
   axis
plt.title('Confusion Matrix test data \n Model - Kneighbors',fontsize
    = 17)      # Set the title
```

Листинг 13 – Код к confusion matrix для всех данных

```
cm = confusion_matrix(y_true, y_pred, labels=[0, 1],  normalize = '
   pred')
print( str(cm) )
disp = ConfusionMatrixDisplay( confusion_matrix = cm,
                                display_labels = ['NO DEFAULT\n 0','
                                   DEFAULT \n 1']
fig, ax = plt.subplots(figsize=(7,7),dpi = 100)
disp.plot(ax=ax)
plt.ylabel('Prediction',fontsize = 13)
plt.xlabel('Actual',fontsize = 13)
plt.title('Confusion Matrix all data \n Model - Kneighbors',fontsize
   = 17)
```

Листинг 14 – Код к confusion matrix для удаленных клиентов

```
y_pred = knn.predict( X_copy[in_new_0_1] ) # Predicted labels of
    removed customers in dataset
y_true = Y_copy[in_new_0_1]                # True labels of removed
    customers in dataset
cm = confusion_matrix( y_true, y_pred, labels=[0, 1],  normalize = '
    pred') # Confusion matrix
# visualization
disp = ConfusionMatrixDisplay( confusion_matrix = cm,
                               display_labels = ['NO␣DEFAULT\n␣0','
                                  DEFAULT␣\n␣1']
                                                                    )
fig, ax = plt.subplots(figsize=(7,7),dpi = 100)
disp.plot(ax=ax)
```

Листинг 15 – Создание модели Decision Tree

```
from sklearn.tree import DecisionTreeClassifier # Import the Decision
    Tree class from the scikit learn library
X_tree = df_copy.loc[:,['LIMIT_BAL',
                        'PAY_0', 'PAY_2',
                        'PAY_3', 'PAY_4',
                        'PAY_5', 'PAY_6']] # Form a matrix of X_trees
                           , which includes only certain columns from
                            df_copy
X_tree = X_tree[~in_new_0_1] # Filter out the customers, excluding
    the suspicious ones
Y_tree = df_copy.loc[~in_new_0_1,['labels']] # Assign class labels 0
    or 1
X_train_tree, X_test_tree, Y_train_tree, Y_test_tree =
    train_test_split( X_tree, Y_tree, test_size = 0.2, random_state =
    0  ) # Split the data into test and training samples
dtree = DecisionTreeClassifier( random_state = 42,
                                max_depth = 10,
                                min_samples_leaf = 2
                               ).fit( X_train_tree, Y_train_tree ) #
                                  Creating and training a model on
                                  training data
dtree.score( X_tree, Y_copy[~in_new_0_1] ) # Calculate model accuracy
    by comparing model predictions to actual class labels
```

Листинг 16 – Удаление ошибочно классифицированных заемщиков

```
X_pay = np.array( X_pay )                          # Conversion to X_pay
    array
X_day = X_pay[:,1:]                                # Extract all columns
    except LIMIT_BAL
index = []                                         # Creating an index list
     to store indexes of remote clients
df_copy['removed_0'] = False                       # Creating the removed_0
     column of unremoved clients
for i,x in enumerate( X_day):
     if x[0]==0 and Y_copy[i]==1:                  # If the client is
        defaulted, but PAY_0 = 0
          index.append(i)                          # The index is added to
             the list
          df_copy.loc[i, ['removed_0']] = True # The client is marked
             as remote
index = np.array(index)                            # The index list is
    converted into an array
index_new = np.where( df_copy['removed_0'].values==False )[0]  #
    Indexes of clients remaining in the sample
X_pay = np.array(df_copy.loc[:,['LIMIT_BAL','PAY_0', 'PAY_2', 'PAY_3'
    , 'PAY_4', 'PAY_5', 'PAY_6']]) # Creating a new sample of features
X_pay[index_new]                                   # Data selection without
     remote clients
df_copy['labels'].values[index_new].shape, len(index) # Output the
    number of remaining clients and the number of removed clients
```